

Hybrid code search via lexical and semantic representations

Luca Stefani (5163797)
luca.stefani.ge1@gmail.com

1 Details of the proposal

1.1 Full specification

This project develops a tool for natural-language code search, combining classical information retrieval with semantic representations to improve natural language navigation of source code.

The system builds a lexical TF-IDF representation from identifiers, signatures, documentation, and code tokens, and combines it with embeddings produced by CodeBERT or UniXcoder.

Python extraction uses the AST while Java extraction uses regular expressions and brace code extraction.

The tooling supports indexing, searching, and benchmarking with hit rate, MRR, recall, and precision.

Compared with the original proposal, the implementation also includes UniXcoder support, a 28-query Python/Java benchmark, persistent indices, and a system that allows other languages to be integrated.

Code can be found at github.com/luca020400/nlp.

1.2 Proposal type and difficulty

This is a creative proposal specifically for the development of software, evaluated as hard.

2 Introduction

Large codebases are difficult to navigate, especially for new developers. A user often knows what they want to do in natural language, but not the exact identifier used in the code.

A query such as “How do I sign out?” may be correct but still fail in a keyword-based search engine if the repository uses `logout`, `endSession`, or `terminateSession` instead of the same words from the query.

This project addresses that problem with a hybrid retrieval pipeline. The lexical side uses TF-IDF to capture exact token overlap between a query and code metadata. The semantic side uses pretrained encoder models to represent code and queries in a dense embeddings space, so that related terms can still match even when the words are different. Finally results will be ranked by combining the lexical and semantic scores.

Each supported language has its own extractor, but both map their results into the same intermediate representation.

This design keeps the project extensible. If support for a new language is needed later, the only missing piece is a new extractor that fills the same IR fields. The rest of the pipeline does not need to change.

2.1 The problem

The main problem is retrieval over source code when the query is written in natural language. A user might search for:

- “login function”;
- “terminate session”;
- “password reset”;
- “update user profile”;
- “delete user account”.

While these queries are clear to a human, they may not match the exact keywords used in the codebase.

Thus introducing another layer, in this case semantic retrieval, can help improve the search results.

2.2 Objectives

The project has four practical objectives:

- Extract function-level code units with their location information.
- Make the extracted code searchable through both TF-IDF and embeddings.

- Expose everything through a small command line interface.
- Provide a benchmark so the models can be compared in the report.

The extractors are lightweight but sufficient for function level indexing. The TF-IDF and embedding representations are computed and stored in a persistent index. The CLI exposes indexing, searching, and benchmarking commands.

To support the report, a small benchmark dataset is included in the repository. It contains 28 queries over 22 functions across six Python and Java files.

3 System overview

The system is organized as a pipeline. From top to bottom:

Component	Role
Extractor layer	Reads Python and Java files and turns them into records.
IR layer	Stores the language-agnostic description of each function.
Lexical index	Computes TF-IDF vectors from identifiers, signatures, comments, and code tokens.
Embedding layer	Computes embeddings with CodeBERT or UniXcoder.
Ranker	Normalizes and blends lexical and semantic scores.
CLI	Exposes indexing, searching, and benchmarking commands.

The flow is “straightforward”. The repository is indexed once and persisted to disk, later on at query time, the query text goes through the same lexical and semantic processing path as the indexed code. The system then scores and ranks the results, returning the top matches to the user, along their source location.

3.1 Intermediate representation

The pipeline is built around the creation of a persistent intermediate representation (IR) of functions. Each function is represented by an identifier, the language name, the file path, the surrounding container (class or module), the symbol name, a readable signature, documentation comments or docstrings, the code body, and the start and end line numbers.

The IR keeps the search pipeline language-agnostic. Python and Java have different syntax, but both become the same type of object after extraction (minus the code body itself).

That makes the next stages of the pipeline “independent” of the source language, simplifying the whole design.

For example, the following Python function:

```
1 def login_user(conn, username, password):  
2     """Authenticate a user."""  
3     ...
```

is translated into the following IR record:

Field	Value
id	python:...:1:login_user
language	python
container	auth
symbol_name	login_user
signature	def login_user(conn, username, password)
docstring	Authenticate a user.
code	full function body
start/end	source line range

The main objective is to create an IR that stores search-relevant information in a uniform way, regardless of the source language. That makes the same structure usable for both the lexical and semantic components of the pipeline.

3.2 Why hybrid retrieval

Lexical retrieval is strong when the query shares vocabulary with the code. If a function contains the words `login`, `session`, or `password`, TF-IDF can pick them up. Semantic retrieval is useful when the query and the code use different vocabulary but have a similar meaning.

For example, a query about “signing out” should still point to a function named `logout_user` or `logoutUser`.

The project combines both approaches treating lexical and semantic search as complementary tools.

TF-IDF is relatively cheap, gives a interpretable result, and good at exact matching. Embeddings given by Transformer models can capture similarities between different words, but they are more expensive to compute and less interpretable.

4 Implementation

The implementation is split into a few small modules.

4.1 Parsing and extraction

Python and Java are handled by separate extractors.

The Python extractor uses the built-in Python AST module to collect function definitions, docstrings, line numbers, and signatures. The Java extractor uses regular expressions for method headers and a brace-aware scanner for method bodies.

This is sufficient for scenarios where the focus is on function level search, as we don't do any "full" program analysis.

The extraction process does not try to parse every possible language feature, it picked a subset of the language that is just enough for function search.

4.2 Lexical indexing

The lexical side uses TF-IDF from scikit-learn. The text passed to the vectorizer is not just raw code. It is a constructed text made from:

- function names;
- fully qualified names;
- signatures;
- docstrings or comments;
- code tokens split from identifiers.

This matters because source code search is sensitive to identifier structure. A function called `login_user` should still be searchable through the terms "login" and "user". The IR decomposes identifiers and exposes those to TF-IDF.

4.3 Semantic encoding

The semantic component supports two pretrained encoders. `codebert` is a RoBERTa-based model trained on paired natural-language and programming-language data, while `unixcoder` is a RoBERTa-based model trained for unified cross-modal code understanding and generation.

Model	Backbone	Training idea	Use
<code>codebert</code>	RoBERTa encoder	NL-PL pretraining	pretrained retrieval baseline
<code>unixcoder</code>	RoBERTa encoder	unified cross-modal pretraining	broader code-text alignment

Both encoders expose the same fixed-size vector interface, but their input preparation is different.

CodeBERT encoding. Code is tokenized with the pretrained CodeBERT tokenizer, which adds sequence boundary tokens and splits identifiers into subtokens. Following the extraction procedure described by Emergent Mind [1], the encoder returns a contextual vector for every token in `last_hidden_state`. The implementation masks padding and special tokens, mean-pools the remaining code-token vectors, and L2-normalizes the resulting 768-dimensional embedding.

UniXcoder encoding. UniXcoder uses the official Microsoft wrapper in encoder mode. The wrapper sets the mode token, builds the corresponding attention mask, and returns the last hidden state. The implementation mean-pools the valid token vectors and L2-normalizes the result.

Both model are exposed through the shared interface ‘`SemanticEncoder`’, so the rest of the code does not care which encoder is in use.

For each function, the input passed to the encoder contains its documentation, signature, and up to the first 2000 characters of its body.

4.4 Ranking

Ranking is done by blending normalized lexical and semantic scores.

The implementation computes both similarities, scales them to a comparable range, and then combines them with a weight `alpha`. The current default is 0.5, which gives the lexical and semantic sides equal influence.

TF-IDF vectors are L2-normalized by the vectorizer, so the dot product between the query vector and each function vector is equivalent to the cosine similarity.

The indexed semantic vectors and the query embedding are also L2-normalized explicitly therefore their dot product is yet another cosine similarity.

These values are the `lexical` and `semantic` scores printed by the CLI.

The hybrid score is:

$$\text{final} = (1 - \text{alpha}) * \text{lexical} + \text{alpha} * \text{semantic}$$

Before this equation is applied, each score array uses fixed zero-anchored scaling: every score is divided by the largest remaining score. Thus a score of zero remains zero, and the largest positive score becomes one.

The CLI displays both “raw” cosine similarities, while the final score is the weighted combination of the normalized values.

As an alternative to score blending, the ranker supports Reciprocal Rank Fusion (RRF) [2]. Instead of normalizing and weighting different the similarity scores, RRF treats retrieval as a voting problem based on the relative rank order.

Each function d receives points according to its placement in the lexical and semantic rankings:

$$\text{RRF}(d) = \sum_{m \in \{\text{lex}, \text{sem}\}} \frac{\mathbb{I}(d \in m)}{k + r_m(d)}$$

where $r_m(d) \geq 1$ is the rank assigned by retriever m , and k is a smoothing constant.

By using reciprocal rank, top-ranked candidates are rewarded exponentially more than lower ones, while the smoothing constant k lowers the advantage of a single extreme rank over consistent matches found across both lists. Functions with no lexical match receive zero points from the lexical term, ties share their average rank, and ties in the final RRF score are decided by their ID order.

4.5 Persistence

The index is saved on disk so the codebase only has to be parsed once. The saved artifacts are:

- `functions.json` for the IR records
- `vectorizer.pkl` for the TF-IDF vocabulary and configuration
- `tfidf.npz` for the sparse lexical matrix
- `semantic.npy` for the dense embeddings
- `metadata.json` for index metadata

5 Command line usage

The project is designed as a CLI tool.

5.1 Installation

The required Python dependencies include NumPy, scikit-learn, and PyTorch. The commands below can be run from the project root.

5.2 Indexing a codebase

To index a repository, run:

```
1 python -m hybrid_search.cli index \  
2   path/to/codebase \  
3   --out artifacts/index \  
4   --encoder unixcoder
```

The command scans supported files, currently `.py` and `.java`, extracts functions, builds the lexical and encoder matrices, and stores the result in the output directory.

5.3 Searching for a function

To search for a query such as “login function”, run:

```
1 python -m hybrid_search.cli search \  
2 --index artifacts/index \  
3 --query "login function" \  
4 --encoder unixcoder
```

The command prints a ranked list of matching functions with their score breakdown and source location. The semantic matrix is created when the index is built, so the index must be searched with the same encoder used during indexing. To compare `codebert` and `unixcoder`, create a separate index for each encoder. The optional `--fusion rrf` argument selects Reciprocal Rank Fusion instead of the default zero-anchored blend.

5.4 Benchmarking

The benchmark command compares the different encoders on the test query set:

```
1 python -m hybrid_search.cli benchmark \  
2 --index benchmarks/index \  
3 --queries benchmarks/queries.json \  
4 --models codebert,unixcoder \  
5 --alphas 0,0.5,1 \  
6 --top-k 3
```

This prints a JSON report for the requested encoders and weights. To recreate the benchmark index first, run:

```
1 python -m hybrid_search.cli index \  
2 benchmarks/sample_codebase \  
3 --out benchmarks/index \  
4 --encoder codebert
```

The benchmark reuses the extracted functions and lexical index, recomputing function embeddings when the encoder changes.

6 Evaluation

6.1 Benchmark setup

The benchmark in `benchmarks/` contains 22 functions across six small Python and Java files and 28 queries in natural language. The queries cover authentication, account handling, cache operations, JSON and CSV processing, file reading, and inventory management. Most queries have only one relevant function, but one has two relevant functions. Queries with no relevant code are excluded from the current testset.

The experiment compares CodeBERT and UniXcoder using the score blending technique described in Section 4.4. The weight is set to $\alpha = 0$ for lexical-only retrieval, $\alpha = 0.5$ for an equal-weight blend, and $\alpha = 1$ for encoder-only retrieval.

All configurations search the same 22 functions and return the top three results. An additional run uses RRF once per encoder with its default smoothing constant $k = 60$.

6.2 Metrics

All metrics are averaged over the 28 queries. For single-answer queries, Recall@ k equals hit rate at the same cutoff; the multi-answer query uses the fraction of its relevant functions retrieved.

Metric	Meaning
Recall@1	Fraction of queries whose relevant function is ranked first.
Recall@3	Fraction of queries whose relevant function appears in the first three results.
MRR@3	Mean reciprocal rank within the top three: $1/r$ if the relevant function occurs at rank $r \leq 3$, and zero otherwise. Stored as <code>mrr</code> in the JSON report.
Precision@3	Average fraction of the three returned results that are relevant.

For example, a relevant function at rank 1 contributes 1 to both recall measures and MRR@3, but only 1/3 to Precision@3. At rank 2, it contributes 0 to Recall@1, 1 to Recall@3, 1/2 to MRR@3, and 1/3 to Precision@3. A result at rank 5 contributes zero to all reported metrics because it falls outside the retrieved top three. For a single-answer query, Precision@3 cannot exceed 1/3; the multi-answer query can contribute more than one relevant result. MRR@3 additionally distinguishes positions within the returned results.

6.3 Results

Table 1 reports the values saved in `benchmarks/results.json`. The lexical-only configuration is shown once because both encoders share the same TF-IDF ranking at $\alpha = 0$.

α	Encoder	Recall@1	Recall@3	MRR@3	Precision@3
0.0	Any	0.84	0.93	0.89	0.32
0.5	CodeBERT	0.84	0.89	0.88	0.31
0.5	UniXcoder	0.88	0.93	0.90	0.32
1.0	CodeBERT	0.04	0.21	0.12	0.07
1.0	UniXcoder	0.63	0.82	0.72	0.29

Table 1: Retrieval on the 28-query benchmark.

TF-IDF reaches Recall@1=0.84 and Recall@3=0.93 on the 28-query benchmark. The CodeBERT blend retains Recall@1=0.84 but falls to Recall@3=0.89,

while the UniXcoder blend reaches Recall@1=0.88 and MRR@3=0.90 with Recall@3=0.93.

The metric values are averages over all 28 queries. The query with two correct functions also contributes a partial recall value when only one of those functions is retrieved.

The aggregate metrics summarize different rankings: UniXcoder has higher Recall@1, Recall@3, and MRR@3 than the CodeBERT blend, while both blends have the same Precision@3 after rounding.

Failures in top three Picking UniXcoder as the best configuration, the following queries fail to fetch the expected function in the top three:

- “need replenishing” – expected `listLowStock`;
- “stop advertisements” – expected `unsubscribe`.

The cache query is a useful rank-one near-miss: `purge_expired_entries` has a good semantic score, but its lexical score is zero. With equal-weight blending, that zero lexical component drags down the final score and leaves it at rank three. The query therefore succeeds at Recall@3 but fails at Recall@1.

RRF comparison. Table 2 reports RRF scores from `benchmarks/results_rrf.json`, using the zero-match and tie handling described in Section 4.4.

Encoder	Recall@1	Recall@3	MRR@3	Precision@3
CodeBERT	0.38	0.66	0.52	0.23
UniXcoder	0.77	0.86	0.82	0.30

Table 2: Unweighted RRF on 28 queries, with $k = 60$.

RRF produces Recall@1 values of 0.38 and 0.77 for CodeBERT and UniXcoder respectively, compared with 0.84 and 0.88 under fixed zero-anchored fusion. Both RRF configurations also have lower MRR@3.

The scope and limitations of these comparisons are discussed in Section 7.

7 Discussion

The results show that the lexical representation is a strong baseline for this benchmark: TF-IDF reaches Recall@1=0.84 and Recall@3=0.93.

The CodeBERT blend retains Recall@1=0.84 but lowers Recall@3 to 0.89, showing that adding a semantic signal can also disturb otherwise useful lexical rankings.

UniXcoder provides more useful information: its blend raises Recall@1 to 0.88 and MRR@3 to 0.90 while keeping Recall@3 at 0.93. This is a minor improvement in early ranking, rather than evidence that semantic retrieval is overall better.

The encoder-only results make that distinction clearer. UniXcoder alone reaches Recall@1=0.63 and Recall@3=0.82, while CodeBERT reaches only 0.04 and 0.21.

On this small search task, removing the lexical signal is expensive, especially for CodeBERT.

RRF is weaker than the scores of both encoders: Recall@1 is 0.38 for CodeBERT and 0.77 for UniXcoder, compared to 0.84 and 0.88 respectively.

Since RRF uses each component’s rank order and gives the lexical and semantic rankings equal weight, a correct lexical match can be weakened by a less useful semantic rank.

In this benchmark, score blending shows the best results, but RRF may be more robust when the two components have very different score distributions.

7.1 Dataset limitations

The benchmark is a small, and the queries sometimes reuse words (or part of them) from identifiers or documentation. It does not represent large repositories, so the results should not be read as general truth.

The searcher also always returns a ranking: detecting an unanswerable query would require thresholding raw scores before normalization, or implementing yet another component with that specific purpose.

7.2 Possible extensions

A possible path to improve the tooling and the benchmark is to:

- adding support for more programming languages with new extractors
- using a larger and more diverse benchmark set
- experimenting with different values of the hybrid weight, weighted RRF, or other techniques as a whole
- fine tune/adapt the encoders on the specific IR format

8 LLM usage policy

Grammarly (AI writing assistant) was used to catch grammatical errors and polish the final report.

LLMs were used to help me come up with some ideas for the entries in the benchmark set.

References

- [1] Emergent Mind: Codebert embeddings (2026), <https://www.emergentmind.com/topics/codebert-embeddings>, accessed 2026-09-10
- [2] Shah, D.: Reciprocal rank fusion (rrf) – how to score results from multiple retrieval methods in rag. Medium (Jul 2024), <https://medium.com/@devalshah1619/mathematical-intuition-behind-reciprocal-rank-fusion-rrf-explained-in-2-mins-0>